

What/Why is a Published Data Source?

- Say you have a datasource and your requirement is to build 10 workbooks using this datasource. Here, your approach should be like Publishing the datasource and use it in your 10 workbooks.
- So, if you refresh the published data once, all the 10 workbooks using this published datasource will have the latest/updated data.
- Publishing data sources can also help you to centralize data management. Meaning it will act as a single source of truth across multiple workbooks where the published data source has been used.
- It also gives a benefit for the business users to do the adhoc analysis or self-service analytics from the Tableau server by connecting it to the published data source.
- Without a Tableau desktop license, you can build your own workbook by using the published datasource in the tableau server and save it in your project. But, you need a web-edit or web-authoring access given to do that in server.
- In addition, when you publish and connect to data on the server, people connecting to the data from Tableau Desktop do not need to install and maintain database drivers on their own computers.
- Your business user no need to install the drivers again to connect to the published datasource. As this will be completed as part of server installation. It will also help to have a centralized driver management.
- When you are publishing a data source, Tableau will save it as a TDS (Tableau Data Source) for live connection or TDSX (Tableau server Data extracted) for extract based connections.

How to create a Published Data source: Screenshots are embedded at the end.

- Open a Tableau desktop
- Connect to the database server which your customer use (Ex: Oracle, SQL, Big Data, etc..) from the datasource page
- Drag the required tables and join them properly using Mapping layer or canvas.
- Or Write a Custom SQL which will suffice all your reporting requirements
- Go to the worksheet from the datasource page.
- Make sure you are pulling the columns(Dimensions/Measures) what you needed from your datasource. If not, you can remove all the unwanted columns. This will be better for the performance of your extract and also it will not confuse others.
- Now you can publish the datasource in three possible ways.
 1. Go to the data pane and right click on your datasource. You will see an option to 'Publish It to Server'.
 2. Go to Server tab and you can find 'Publish a Datasource'.
 3. Using a Batch script that uses TABCMD.

- While publishing it will ask for the Project, Name of the datasource, Refresh schedule (If it's an extract based), Authentication details, etc.
- If your datasource connection is in Live mode, then it will publish it to your server and save as TDS file extension.
- If your datasource connection is in extract mode, then it will publish it to your server and save as TDSX file extension.

How to use a Published Data source:

You can use it in two different ways.

1. If you have a Tableau Desktop license, you can open a new Tableau session and in connect to data window select Tableau server. It will list you all the published data sources in your tableau server. You need to choose the one which you want to use.
2. You don't have a Desktop license. But, you have a server access with permissions to connect to Published datasource and also permissions to do the web-edit/Web-authoring in server. You can select the datasource which you want to use and create a new workbook.

Who can Publish a Data Source?

- Any developer who is having Publisher rights to the Tableau server project he/she is publishing.

How to edit a Published Data source:

- You should have your workbook which you developed your Table mappings/Custom SQL and published it to the server. Edit the SQL/Mappings and then re-publish again to server.
- Otherwise, Download the datasource from your server (TDS/TDSX) and then open with your Tableau desktop to see the mappings/SQL and then re-publish it back to the server.

When I edit a Published Datasource do I need to re-publish all my workbooks using it?

- Not necessarily. If it is changing the structure of the data, you are using in your workbook. Then yes, you have to re-publish by doing the required changes in your workbook.
- Scenario 1: I modified one column name from AAA to BBB which were used in four of the ten workbooks using this datasource. Here, after publishing the updated datasource to the server. We need to open all the four workbooks affecting this change and make sure nothing breaks and re-publish it to the server.
- Scenario 2: I edited the PDS (Published Data Source) to add one more filter in the where clause and published it to the server. Do we need to re-publish the workbooks? No, not required. Your workbooks will be connected (Live) to your published data source. So the changes will be affected to the workbooks automatically. What if your workbooks connected in Extract (mode) to your PDS? In this case you need to extract/refresh the data one more time to make the filters change data affect in your workbook.

What is the deployment process for Published Datasource?

Development: First we should publish the Datasource to the Tableau server and then publish your workbooks using the PDS.

QA Environment:

- a. Open the workbook/TDS/TDSX which you used to publish the datasource to Development.
- b. Select the server tab and login to your QA Environment.
- c. Follow the same steps as you did to publish your datasource in Development environment.
- d. Datasource name should be same as Dev
- e. Open the workbooks which you developed using the published datasource.
- f. Go to Data pane, Select the datasource, Right click on it, Choose Tableau Data server, choose edit server and site path
- g. Sign in to QA server, if you are not already.
- h. Choose the datasource from the dropdown.
- i. Make sure the connection name matches and consistent across the environments.

PROD Environment:

- a. Open the workbook/TDS/TDSX which you used to publish the datasource to Development/QA Environment.
- b. Select the server tab and login to your PROD Environment.
- c. Follow the same steps as you did to publish your datasource in Development/QA environment.
- d. Datasource name should be same as DEV/QA
- e. Open the workbooks which you developed using the published datasource.
- f. Go to Data pane, Select the datasource, Right click on it, Choose Tableau Data server, choose edit server and site path
- g. Sign in to PROD server, if you are not already.
- h. Choose the datasource from the dropdown.
- i. Make sure the connection name matches and consistent across the environments.

What if I changed the Database Server A to Server B?

- You have an option to edit the connection even for Published datasource. We can simply edit the connection and point to the Database Server B.
- If your workbooks are using Extract connection of a TDSX. Ideally, it shouldn't be a problem. But, please ensure both the datasource and workbook connections are pointing to Database Server B and you are good.

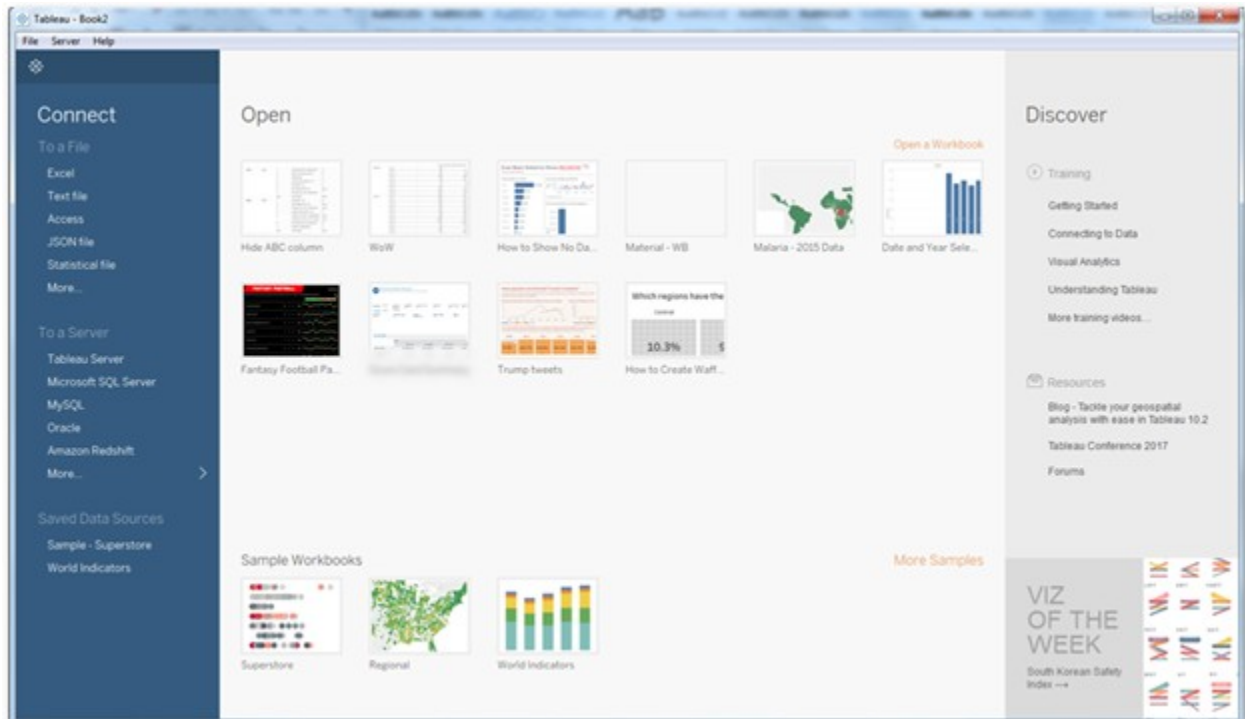
Difference Between Publishing data separately and embedded in workbooks [From KB Article]

Published separately	Embedded in workbook
Publishing data sources centralizes data management, enables policies around “certified” data and governance, and can help to minimize data source proliferation.	Each embedded data source has a disparate connection to the data.
	Each has the potential to show something different than the other at any given time (and data source proliferation is common).
Meant to be shared; becomes available for other Tableau users to connect to.	Data is available only inside the workbook; it is not available for other Tableau Desktop users to connect to.
Extracts can be refreshed on a schedule. You set up one refresh schedule for the extract, and all workbooks that connect to it always show the most current data.	Embedded extracts that aren't refreshed can be useful for showing snapshots in time.
	If you want to keep the data fresh, each workbook must have its own refresh schedule.
Generally helps you to optimize performance on the server or site.	Performance might be affected when the server contains multiple workbooks that connect to the same original data, and each workbook has its own refresh schedule.

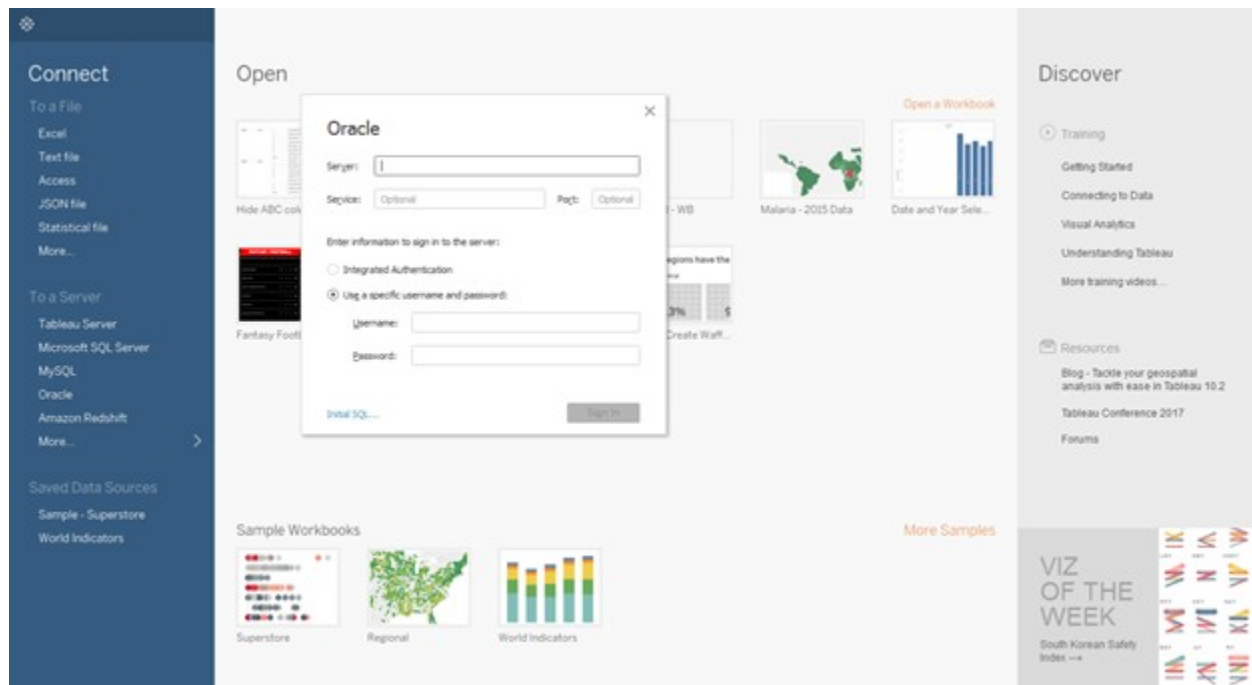
Note: All the points which I am sharing here is truly based on my experiences and knowledge what I have gained. Please correct me if I stated anything incorrectly.

Screenshots:

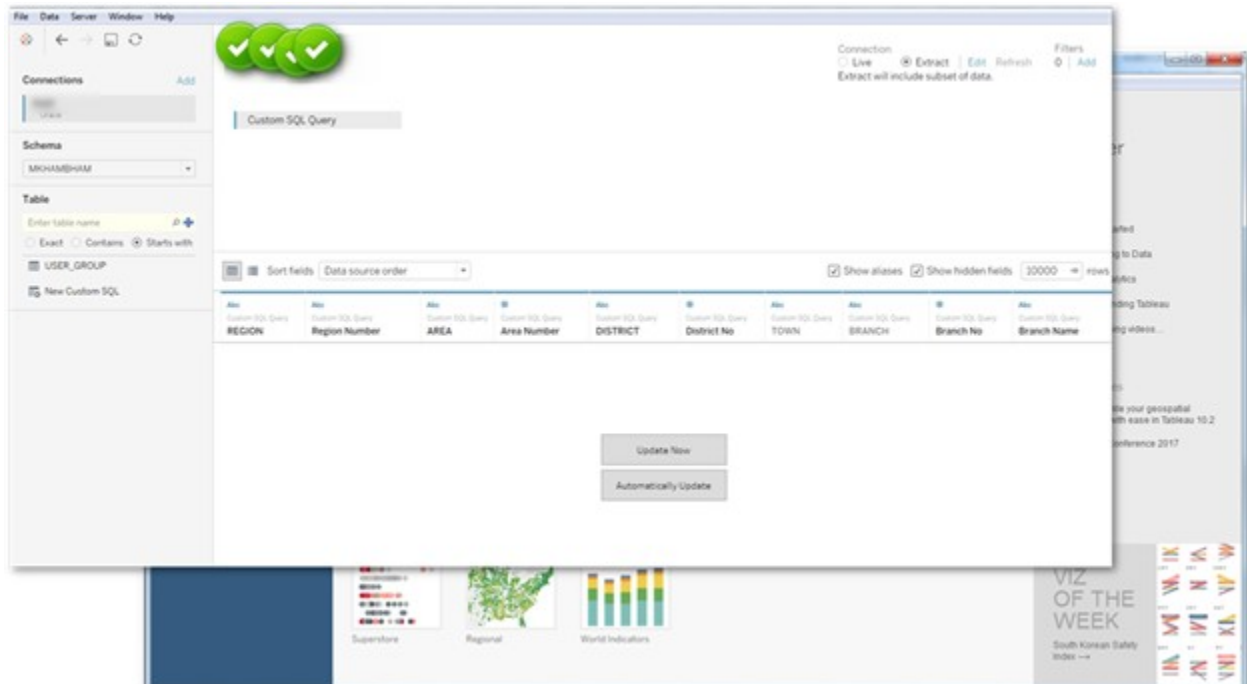
Open a Desktop



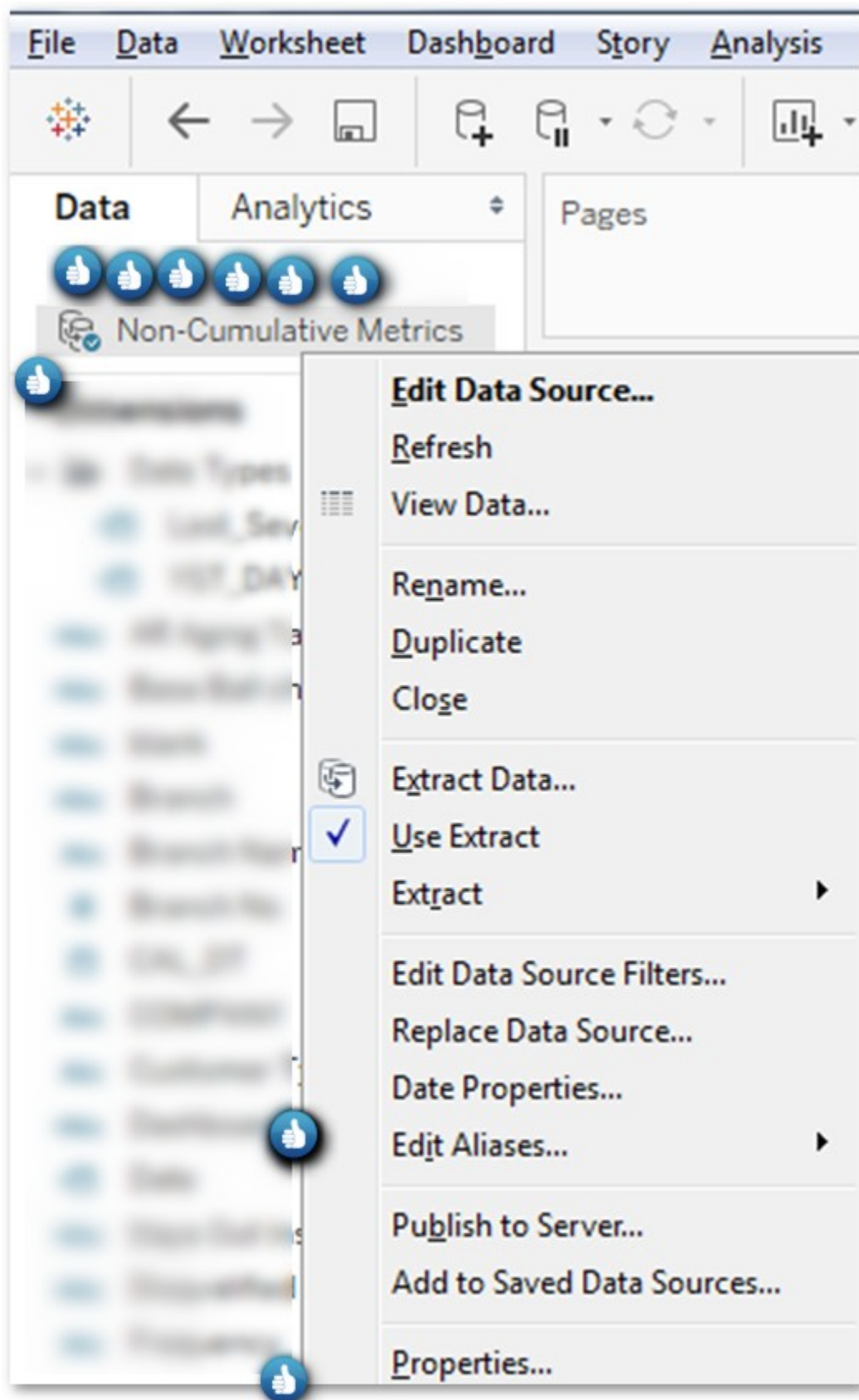
Go to the Datasource connection pane connect to the Data base:



Write the Custom SQL or use your Table joins/Mapping Out of the box tableau in-built feature



Go to the worksheet and publish the datasource to server



Publish Data Source to Tableau Server

Project

Default

Name

Non-Cumulative Metrics

Description

Tags

Add

Refresh Schedule (Full Extract)

None

Permissions
Permissions are locked by the site admini... View

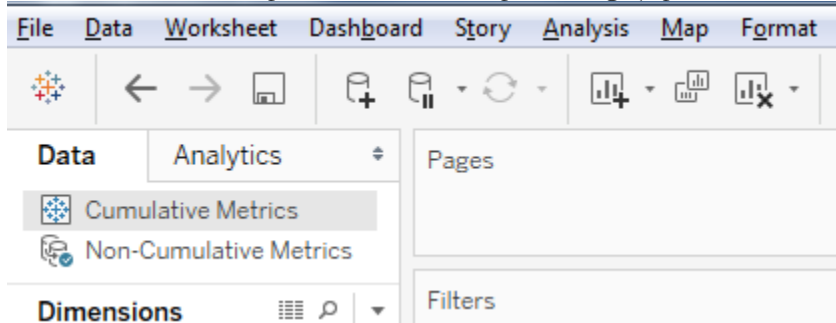
Authentication
Refresh not enabled Edit

More Options

☒ Update workbook to use the published data source

Publish

Once you published you have an option to automatically use the published data source . If you see the above screenshot there is an option to select while publishing. (Update workbook to use the Published datasources)



The screenshot shows the Tableau Desktop interface with the 'Data Sources' tab selected. The interface includes a top navigation bar with 'Home', 'Workbooks', 'Views', 'Data Sources', 'Permissions', and 'Details'. The 'Data Sources' tab is active, displaying a list of data sources. The list has columns for 'Name', 'Views', 'Workbooks', and 'Last extract'. There are five data sources listed, each with a 'Data Source' icon and a 'Last extract' date. The first two data sources are 'Sales' and 'Marketing', both with 11 views and 3 workbooks. The next two are 'Sales - Cumulative Metrics' and 'Marketing - Cumulative Metrics', both with 4 views and 0 workbooks. The last one is 'Sales - Non-Cumulative Metrics' with 0 views and 0 workbooks. The 'Last extract' dates are Feb 5, 2017, 24:00, Feb 5, 2017, 25:00, Feb 14, 2017, 1:00, Feb 14, 2017, 1:00, and Jan 29, 2017, 6:00 respectively.

